

Modelos Generativos Profundos

Clase 18: Score matching (parte I)

Fernando Fêtis Riquelme

Otoño, 2026

Facultad de Ciencias Físicas y Matemáticas
Universidad de Chile

Modelos basados en energía

Langevin sampling

Modelos basados en energía

Parametrizar directamente una distribución $p_{\theta}(x)$ usando una red neuronal requiere imponer condiciones propias de una función de densidad:

- **Positividad:** $p_{\theta}(x) \geq 0$ para todo $x \in \mathbb{R}^D$.
- **Normalización:** $\int_{\mathbb{R}^D} p_{\theta}(x) dx = 1$

La primera condición se puede imponer fácilmente usando funciones de activación como ReLU.

En el caso discreto, la segunda condición se puede imponer usando la función softmax. Sin embargo, en el caso continuo esta cantidad es intratable de evaluar en general.

Formulación

Los **modelos basados en energía (EBMs)** reparametrizan la distribución conjunta $p_\theta(x)$ como

$$p_\theta(x) = \frac{\exp(-E_\theta(x))}{Z_\theta},$$

donde $E_\theta : \mathbb{R}^D \rightarrow \mathbb{R}$ es aprendida por una red neuronal, mientras que $Z_\theta = \int_{\mathbb{R}^D} \exp(-E_\theta(x)) dx$ es la cte. de normalización.

- La red neuronal E_θ (**función de energía**) ahora es irrestricta.
- La dificultad se traslada al cálculo de Z_θ (**función de partición**).

En consecuencia, la verosimilitud $\log p_\theta(x)$ sigue siendo intratable. Sin embargo, se verá una forma de entrenar E_θ por máxima verosimilitud (aproximada) sin calcular la constante $Z_\theta > 0$.

Entrenamiento por máxima verosimilitud

Calcular $\nabla_{\theta} \log p_{\theta}(x)$ requiere calcular Z_{θ} , por lo que entrenar E_{θ} por máxima verosimilitud (exacta) es intratable en general.

Sin embargo, este gradiente se puede **estimar** eficientemente
En efecto, se prueba que:

$$\nabla_{\theta} \log p_{\theta}(x) = \mathbb{E}_{p_{\theta}(x)} [\nabla_{\theta} E_{\theta}(x)] - \nabla_{\theta} E_{\theta}(x)$$

- $\mathbb{E}_{p_{\theta}(x)} [\nabla_{\theta} E_{\theta}(x)]$ se estima usando estimaciones de Monte Carlo.
- $\nabla_{\theta} E_{\theta}(x)$ se calcula por diferenciación automática.

Por lo tanto, solo se necesita poder generar muestras desde $p_{\theta}(x)$ para poder entrenar la red neuronal E_{θ} y maximizar $\log p_{\theta}(x)$.

Langevin sampling

Se necesita poder generar muestras desde $p_{\theta}(x) = \frac{\exp(-E_{\theta}(x))}{Z_{\theta}}$.

Dado que Z_{θ} es intratable, se deben usar métodos tipo **Markov chain Monte Carlo (MCMC)** para generar muestras desde $p_{\theta}(x)$.

- Este tipo de algoritmos permite generar muestras desde distribuciones donde solo se conoce una cantidad proporcional a la función de densidad.
- Este es el caso de los EBMs, donde $p_{\theta}(x) \propto \exp(-E_{\theta}(x))$.
- El método MCMC más común en los EBMs es **Langevin sampling**.

El **algoritmo de Langevin** comienza con una muestra $x_0 \sim p(x_0)$ cualquiera (generada, p.g., desde una gaussiana o una uniforme). Luego, realiza la siguiente iteración hasta la convergencia:

$$x_{t+1} = x_t + \frac{\epsilon}{2} \nabla_{x_t} \log p_{\theta}(x_t) + \sqrt{\epsilon} z_t, \quad \text{donde } z_t \sim \mathcal{N}(0, I_D).$$

Este método puede ser visto como una especie de gradiente ascendente sobre $\log p_{\theta}(x)$, con un término adicional de ruido:

- La constante $\epsilon > 0$ actúa como una tasa de aprendizaje.
- El ruido adicional $\sqrt{\epsilon} z_t \sim \mathcal{N}(0, \epsilon I_D)$ evita que la recursión converja a una moda de la distribución $p_{\theta}(x)$.

La iteración de Langevin sampling es:

$$x_{t+1} = x_t + \frac{\epsilon}{2} \nabla_{x_t} \log p_{\theta}(x_t) + \sqrt{\epsilon} z_t, \quad \text{donde } z_t \sim \mathcal{N}(0, I_D).$$

Notar que el gradiente $\nabla_x \log p_{\theta}(x)$ que necesita esta dinámica ahora es con respecto a x y no con respecto a θ .

En particular, para un EBM $p_{\theta}(x) = \frac{\exp(-E_{\theta}(x))}{Z_{\theta}}$, se puede obtener este gradiente por diferenciación automática:

$$\nabla_x \log p_{\theta}(x) = -\nabla_x E_{\theta}(x)$$

Resumen EBM

En resumen, un EBM parametriza $p_\theta(x) = \frac{\exp(-E_\theta(x))}{Z_\theta}$ y es entrenado por máxima verosimilitud aproximada:

- $E_\theta : \mathbb{R}^D \rightarrow \mathbb{R}$ se entrena por gradiente ascendente usando que

$$\nabla_\theta \log p_\theta(x) = \mathbb{E}_{p_\theta(x)} [\nabla_\theta E_\theta(x)] - \nabla_\theta E_\theta(x)$$

- $\mathbb{E}_{p_\theta(x)} [\nabla_\theta E_\theta(x)]$ se estima mediante Monte Carlo usando muestras $x^1, \dots, x^K \sim p_\theta(x)$ generadas mediante Langevin Sampling:

$$x_{t+1} = x_t + \frac{\epsilon}{2} \nabla_{x_t} \log p_\theta(x_t) + \sqrt{\epsilon} z_t, \quad \text{donde } z_t \sim \mathcal{N}(0, I_D).$$

- El gradiente $\nabla_{x_t} \log p_\theta(x_t)$ se calcula por diferenciación automática:

$$\nabla_{x_t} \log p_\theta(x_t) = -\nabla_{x_t} E_\theta(x_t)$$

En la próxima clase.

- Score matching.
- Denoising score matching.
- 5º control de lectura.

Modelos Generativos Profundos

Clase 18: Score matching (parte I)